

第13讲

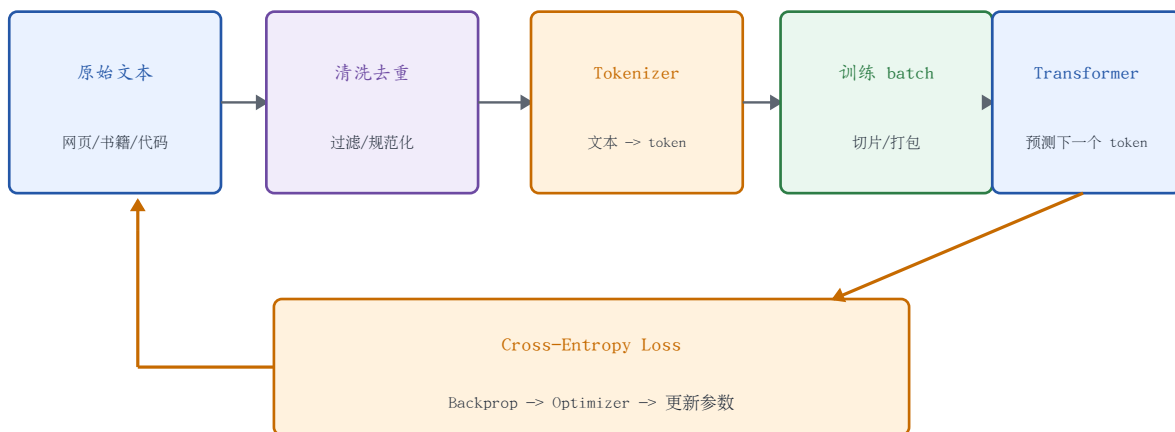
Pretraining：大语言模型到底怎样预训练出来？

从海量文本、next-token prediction 到 Base Model

本讲目标

学完后，你应该能完整解释：原始文本怎样变成训练 token；为什么一个普通句子能产生大量监督信号；Cross-Entropy、Backprop、Optimizer 如何不断修改参数；为什么训练完成后得到的是 Base Model 而不是聊天助手；以及预训练为什么既可能形成泛化能力，也可能产生记忆与数据污染问题。

Pretraining 的完整闭环



这个循环会重复极多次。所谓“预训练”，本质上就是用海量 token 不断产生监督信号。

先记一句

预训练不是“把互联网存进模型”。它是把海量 token 变成一次又一次“预测下一个 token”的训练题，通过损失函数和梯度，把统计规律逐步写进参数。

1. 什么叫 Pretraining（预训练）？

Pretraining 可以拆成两个词：Pre + Training。它表示在模型真正用于问答、对话、工具调用之前，先用大规模通用数据进行第一阶段训练。

阶段	输入	目标	典型产物
Pretraining	海量通用文本/代码等	学习语言与数据分布	Base Model
Instruction Tuning	指令-回答样本	学会按指令回答	Instruction Model
Preference / RL 等	偏好、奖励或反馈数据	进一步塑造行为	更对齐的助手模型

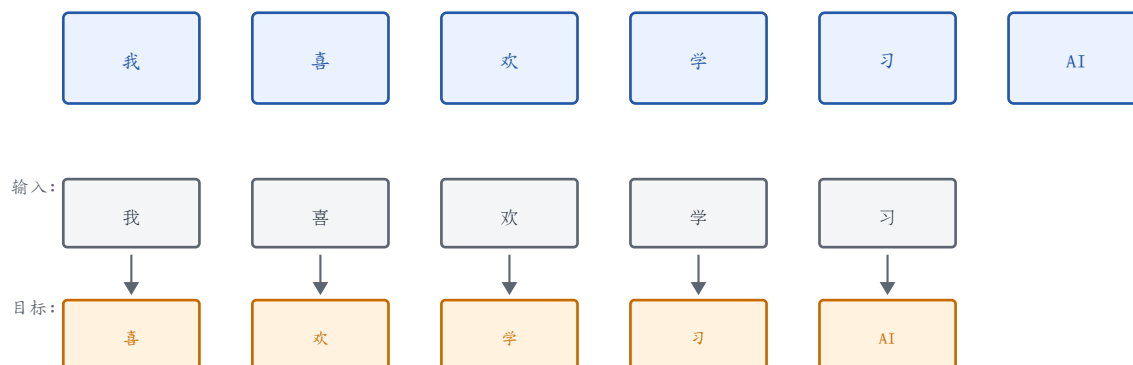
最关键区别

预训练主要解决“模型会不会建模语言和世界中的统计结构”；后续对齐主要解决“模型是否按用户希望的方式响应”。

2. 从第12讲接过来：GPT 预训练到底在做什么？

第12讲已经知道 GPT 类模型采用 Decoder-only，并通过 Causal Mask 预测下一个 token。预训练就是把目标放大到极大量数据、极多训练 step 上。

一个序列如何同时制造多个 next-token 训练目标



实现上通常是“输入序列与目标序列错开一位”（shift by one）。

直觉

你可以把互联网文本想象成一条极长的“出题机”：每向右移动一个位置，就产生一道新的 next-token prediction 题。

3. 一个句子为什么能同时产生很多训练题？

假设序列为：我 喜欢 学习 人工 智能。模型并不是只在最后预测一次，而是在多个位置上同时学习：

模型可见前缀	训练目标
我	喜欢
我 喜欢	学习
我 喜欢 学习	人工
我 喜欢 学习 人工	智能

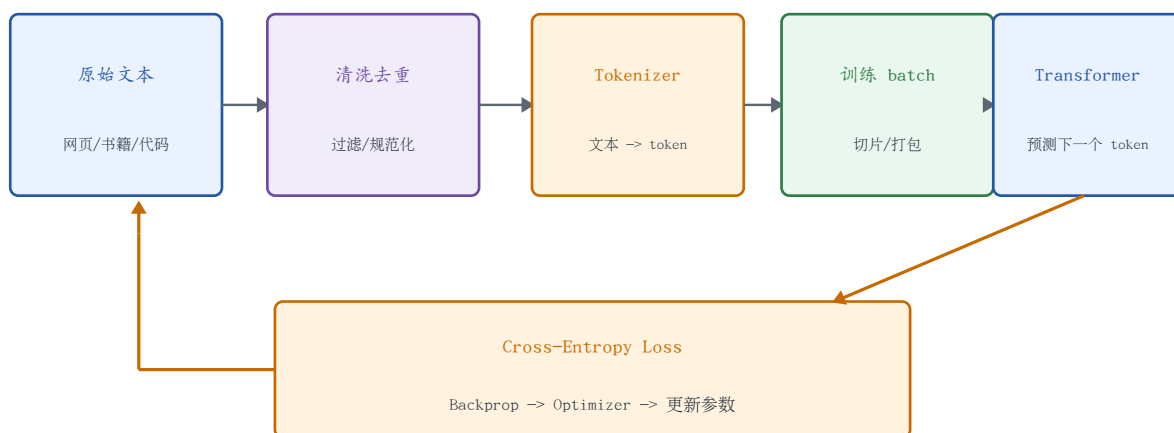
训练实现中常把输入 token 与目标 token 错开一位，这就是常见的 shifted targets。Causal Mask 保证每个位置不能偷看自己的未来答案。

为什么训练效率高

虽然任务逻辑是“一个 token 接一个 token”，但训练时整段正确文本已经存在，因此 GPU 可以同时计算多个位置的 loss。

4. 文本进入训练前，先经历什么？

Pretraining 的完整闭环



这个循环会重复极多次。所谓“预训练”，本质上就是用海量 token 不断产生监督信号。

- 收集原始语料：网页、书籍、代码、论文、论坛、问答等来源；
- 数据清洗：删除明显垃圾、解析失败内容、异常编码等；
- 去重：减少大量重复网页或模板文本；
- 质量与安全过滤：根据训练目标剔除不需要的数据；
- Tokenizer：把文字转成 token IDs；
- 切片 / Packing：把 token 组织成固定或近似固定长度的训练序列。

注意

不同模型的数据配方差异很大，而且很多商业模型不会公开完整训练数据。这里学习的是通用流水线，而不是某一家模型的具体配方。

5. Tokenization 在预训练流水线里扮演什么角色？

神经网络不能直接读网页文字。Tokenizer 先把字符串切成 token，再映射成 Token IDs。之后 Embedding 才把 ID 变成向量。

文本 -> Tokenizer -> Token IDs -> Embedding -> Transformer

- 词表决定模型的基本离散符号单位；
- 同样的字符，不同 tokenizer 可能产生不同 token 数；
- token 数直接影响训练数据规模、上下文长度和计算成本；
- 代码、中文、数字等内容的切分方式也会影响学习效率。

与第2讲衔接

第2讲学 Token 是为了理解“模型看见什么”；这一讲要进一步理解：预训练的计量单位通常也是 token，而不是“网页篇数”。

6. Packing / Sequence 是什么？为什么不能一篇文章一个 batch？

GPU 更擅长处理规则张量。训练时常把很多文档切成或拼成长度较接近的 token 序列，例如长度为 2K、4K 或更长的片段。

问题	直接按原文档训练可能怎样	Packing 思路
文档长短差异大	大量 padding，浪费算力	把多个短片段拼入同一训练序列
超长文档	单条样本放不下	切成多个窗口
batch shape 不规则	GPU 利用率低	尽量形成规则 tensor

一个细节

拼接多个文档时，需要正确处理文档边界、特殊 token 和 attention 规则，否则模型可能把无关文档错误地当成连续文本。

7. 模型在一个训练 step 中具体做什么？

- 1) 取一个 batch 的 token IDs;
- 2) 做 Embedding, 并通过多层 Transformer;
- 3) 每个位置输出整个词表的 logits;
- 4) 用正确的“下一个 token”计算 loss;
- 5) 对 loss 做反向传播, 得到每个参数的梯度;
- 6) Optimizer 根据梯度更新参数;
- 7) 进入下一 batch, 继续循环。

可以把一次训练 step 写成:

```
tokens -> forward -> logits -> cross_entropy -> backward -> optimizer.step()
```

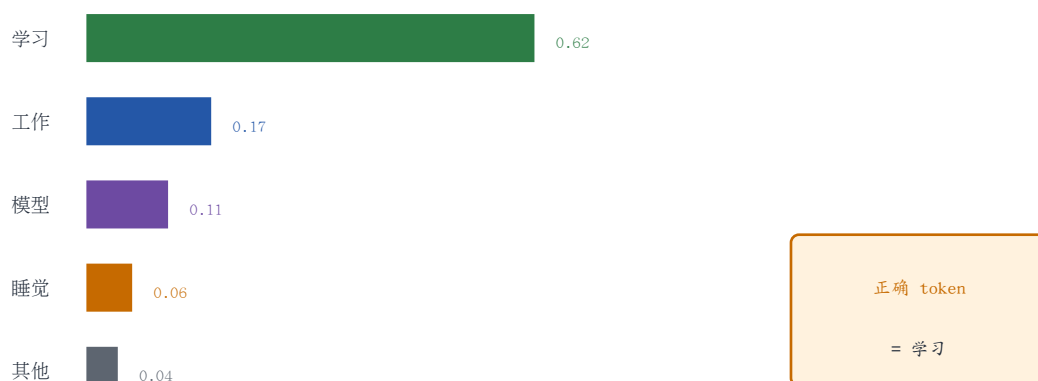
真正的“学习”发生在哪里？

不是发生在数据文件里, 而是在 optimizer

更新参数的那一刻。数据只是不断提供“应该怎样修改参数”的训练信号。

8. Logits、Softmax、Probability 再复习一次

模型不是直接输出“答案”，而是输出整个词表的分数



Cross-Entropy 会惩罚“正确 token 概率太低”，鼓励模型把概率质量移动到正确答案上。

模型最后输出的是 logits，也就是对词表中每个 token 的未归一化分数。Softmax 把这些分数变成概率。训练目标要求正确 token 的概率尽量高。

不要误解

训练通常不是“如果预测错了才修改”。即使正确 token 已经排第一，只要概率分布还可以更符合目标，loss 仍能产生梯度。

9. Cross-Entropy 为什么适合 next-token prediction?

如果正确 token 的预测概率是 p ，那么单个位置的负对数似然可以直观写成：

$$\text{loss} = -\log(p_{\text{correct}})$$

正确 token 概率	$-\log(p)$	直觉
0.90	约 0.105	模型很自信且正确，惩罚小
0.50	约 0.693	不够确定
0.10	约 2.303	正确答案概率很低，惩罚大
0.01	约 4.605	非常差，惩罚更大

本质

Cross-Entropy 让模型学习“把概率质量放到真实下一个 token 上”，而不是只学习一个硬性的对/错标签。

10. 一个 batch 的 Loss 是怎样来的？

一个 batch 往往包含很多序列，每个序列又包含很多可预测位置。因此实际训练 loss 通常是大量 token loss 的平均或加权聚合。

```
batch loss = mean(loss_1, loss_2, ..., loss_N)
```

- Padding 位置通常不参与 loss;
- 文档边界和特殊 token 是否参与，要看训练设计;
- 不同数据来源有时会设置不同采样权重;
- loss 数值下降只说明训练目标改善，不自动等于“所有能力都更强”。

科研习惯

看到论文报告 training loss / validation loss 时，先问：在哪些 token、什么数据、什么 masking 和 weighting 规则下计算？

11. Teacher Forcing：训练时为什么不像真实生成那样一步一步？

训练 vs 生成：最容易混淆的一点



训练时把真实前缀作为条件，常被称为 teacher forcing；它提高训练效率，但会带来训练-生成分布差异。

训练时，模型的历史前缀通常来自真实数据，而不是来自自己刚刚生成的错误 token。这让训练可以稳定且并行。这个思想通常称为 teacher forcing。

潜在差异

部署生成时，模型必须吃自己刚生成的 token；如果早期一步生成偏了，后面上下文也随之改变。这是训练分布与生成分布不完全一致的来源之一。

12. Backpropagation 在大模型预训练里到底反传什么？

Loss 是一个标量，但模型可能有数十亿参数。反向传播通过链式法则计算：如果某个参数稍微变化，Loss 会怎样变化。

```
gradient_i = dLoss / d(parameter_i)
```

- 靠近输出层的参数得到直接的误差信号；
- 误差信号继续穿过 FFN、Attention、Norm、Embedding 等层反传；
- 最终每个可训练参数都会获得一个梯度；
- 优化器再根据这些梯度更新参数。

与第5讲衔接

第5讲的梯度下降并没有因为模型变成 LLM 而改变本质；只是参数更多、计算图更深、分布式系统更复杂。

13. Optimizer 做什么？为什么通常不是最简单的 SGD？

优化器决定如何把梯度变成参数更新。大模型训练常使用 Adam/AdamW 一类优化器及其变体，因为它们会利用梯度的一阶、二阶统计来调节每个参数的更新步幅。

概念	直觉
Learning Rate	这一步整体迈多大
Warmup	训练初期先把步幅逐渐拉起来
Decay Schedule	后期逐渐降低学习率
Gradient Clipping	限制异常大梯度
Weight Decay	对参数规模施加一定约束

注意

真实训练配方是一个系统工程问题。学习阶段先理解“梯度只是方向信息，优化器决定怎样走”。

14. 为什么预训练需要极大算力？

成本来自三个同时增长的量：模型参数量、训练 token 数、每个 token 的计算量。一个 token 需要穿过很多层，每层又包含大规模矩阵乘法。

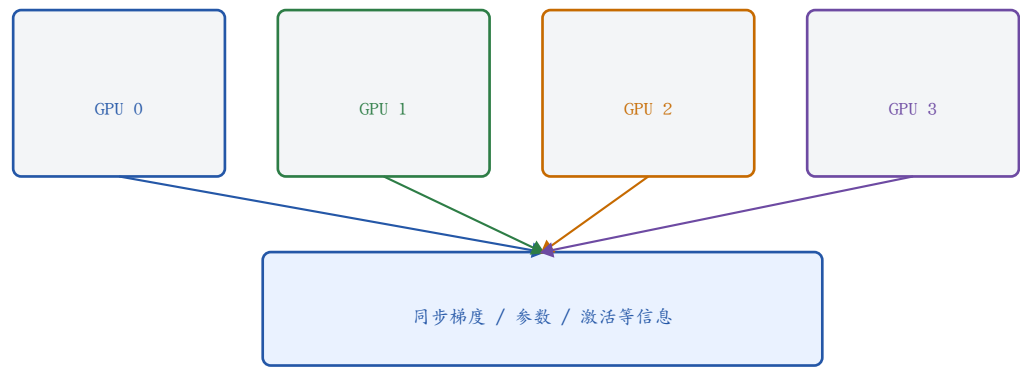
- Forward: 计算隐藏表示和 logits;
- Backward: 还要保存/重算中间激活，并计算梯度;
- Optimizer: 更新海量参数和优化器状态;
- 通信: 多 GPU / 多节点之间还要同步数据。

所以训练比推理重得多

推理通常只做 forward; 预训练不仅要 forward, 还要 backward + optimizer, 而且要在极大量 token 上重复。

15. 单张 GPU 放不下怎么办？

为什么大模型预训练需要分布式训练？



不同方案可能切数据、切参数、切层或切序列；核心目标是把单卡放不下、算不完的训练拆开。

思路	简单理解
Data Parallel	不同 GPU 处理不同 batch，之后同步梯度
Tensor Parallel	把一层的大矩阵切到多张 GPU
Pipeline Parallel	把不同层放在不同 GPU / stage
Sequence / Context Parallel	把长序列维度也拆开处理

你现在不需要实现这些

但以后读 AI Systems / LLM Systems 论文时会频繁遇到。预训练本身就是一个巨大的分布式系统问题。

16. Checkpoint 是什么？为什么训练不能“从头到尾一口气跑完”？

训练周期可能很长，硬件也可能故障，因此系统会定期保存

Checkpoint。它通常包括模型参数，训练时还可能保存优化器状态、学习率状态、随机数状态等。

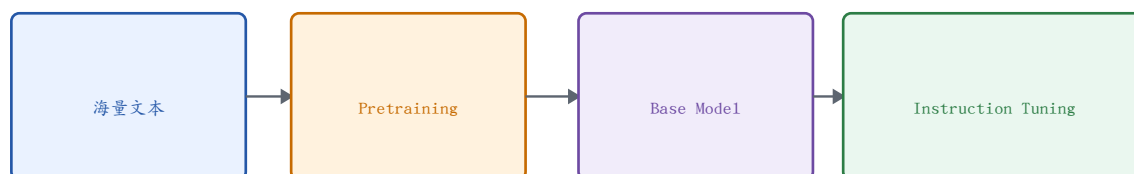
- 发生故障后可以从最近 checkpoint 恢复；
- 可以保留不同训练阶段的模型做评测；
- 也可能基于某个 checkpoint 继续做指令微调或领域训练；
- checkpoint 文件很大，因此保存和加载本身也是系统成本。

术语辨析

Model checkpoint 是训练状态的快照；Model release 往往还涉及权重格式、Tokenizer、配置文件、许可证等。

17. 预训练结束后得到什么？Base Model 是什么？

Pretraining 结束后得到的不是“聊天助手”，而是 Base Model



之后还可能继续做偏好对齐 / RL / 安全训练等

Base Model 最核心的能力仍是：给定前缀，对后续 token 建模。

Base Model 已经能续写文本，往往也具备大量知识和模式能力，但它未必会像聊天助手那样稳定地遵循“请总结”“请用表格回答”等指令。

关键区别

会语言建模 $\neq$ 会当助手。“请按要求回答”的行为通常需要后续 Instruction Tuning / Preference Alignment 等阶段继续塑造。

18. 为什么“预测下一个 token”会学到语法？

要把下一个 token

预测好，模型必须利用上下文中的规律。比如主谓一致、语序、标点、固定搭配都会影响后续 token 的概率。

要预测的现象	模型可能需要捕捉的规律
The dogs are ...	主语复数与动词形式
如果 A 大于 B, B 大于 C, 那么 ...	关系链与逻辑模式
def foo(x): ...	代码语法、缩进、API 模式
法国的首都是 ...	事实共现与知识模式

重要但别神化

训练目标确实很简单，但数据分布本身非常复杂。要持续降低 loss，模型就被迫学习越来越多能帮助预测的结构。

19. 为什么会学到“知识”？参数里到底存了什么？

模型不会把每个网页以数据库记录的形式直接存储。训练过程中，频繁出现的实体关系、语言模式和事实关联会共同影响权重。最后形成的是一个高维、分布式的参数表示。

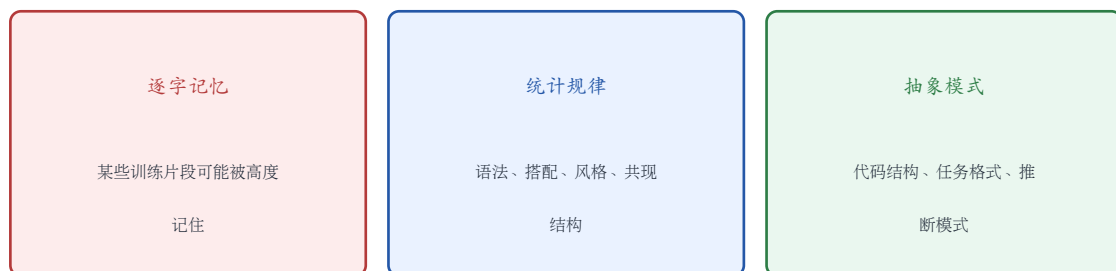
- 某些知识可能分散在大量参数和层中；
- 某些训练片段也可能被高度记忆；
- 同一事实可能从许多不同文本中反复获得训练信号；
- 模型调用知识时仍然是通过当前上下文计算概率，不是传统数据库检索。

类比

更像一个人读了大量材料后形成“脑中模式”，而不是把所有书页逐张装进文件柜；但这个类比也不意味着模型和人脑机制相同。

20. Memorization 与 Generalization：不是二选一

模型学到的东西：记忆、统计规律与可组合能力同时存在



因此“模型只是背训练数据”与“模型完全不记忆训练数据”都过于简单。

大型语言模型既可能记住训练片段，也能对没有逐字见过的组合进行泛化。二者可以同时存在。

为什么科研上重要

数据泄漏、benchmark contamination、版权记忆、隐私泄露，都与“模型究竟记住了什么”有关；而科研价值又常常依赖模型能否在未见任务上泛化。

21. “模型只是压缩互联网”这句话对不对？

这是一种有启发性的比喻，但不能字面理解。参数确实以有限容量吸收了数据中的统计结构，但训练不是普通 ZIP 压缩，模型也不能无损还原全部训练数据。

可以保留的直觉	不能过度推导
有限参数承载大量数据规律	所以模型必然逐字保存所有网页
预测目标鼓励建模可复用模式	所以模型内部就是传统数据库
重复、高频结构更容易影响参数	所以低频事实一定完全学不到

科研习惯

遇到“LLM 就是 X”这种一句话定义时，先把它当作解释视角，而不是严格等价。

22. Scaling：为什么数据、参数、算力要一起考虑？

如果模型很大但数据太少，容易训练不足或浪费容量；如果数据巨大但模型太小，也可能容量不足。训练设计会在模型规模、token 数和计算预算之间寻找平衡。

- Model size：能容纳和处理多复杂的模式；
- Data / tokens：提供多少训练信号；
- Compute：实际执行多少矩阵计算；
- Optimization：这些计算能否稳定转化为更低 loss。

先埋一个伏笔

后续会专门讲 Scaling Law。现在先记住：LLM

变强不是简单“只加参数”，而是数据、算力、优化与架构共同作用。

23. 为什么 Loss 下降不等于“模型全面变聪明”？

Training loss 是在特定数据分布、特定 next-token

目标上的平均指标。它下降是必要信号之一，但不能自动推出所有下游能力都改善。

- 训练数据里高频模式可能主导平均 loss;
- 某些稀有能力对总体 loss 贡献很小;
- 长上下文、推理、安全、工具使用等能力可能需要专门评测;
- 训练 loss 降低也可能伴随记忆、偏差或某些行为副作用。

与科研评测衔接

这就是为什么论文不能只看 training loss, 还要设计 held-out validation、downstream benchmark、OOD、robustness 等评测。

24. 数据污染为什么会让 Benchmark 看起来“虚高”？

如果 benchmark 题目或高度相似内容已经出现在预训练语料中，模型的高分可能部分来自记忆或熟悉，而不完全代表对新问题的泛化。

情况	风险
题目逐字出现在训练集	最明显的 contamination
题目改写但答案结构高度相似	仍可能泄漏
评测网站被公开抓取	未来模型可能见过
开发集被反复人工调 prompt	也会产生评测过拟合

与你以后做 Agent Benchmark 直接相关

Agent 系统不仅可能污染知识题，还可能见过 benchmark 的网页、代码、任务模板或标准答案。评测设计必须考虑这一点。

25. 小实验：亲手训练一个极小的 next-token 模型

材料包中的 `04_toy_bigram_pretraining.py` 只用 NumPy。它不会训练 Transformer，而是训练一个最小 Bigram Language Model：根据“当前 token”预测“下一个 token”。

- 构造一个很小的中文/英文混合 token 序列；
- 建立 transition logits 参数矩阵；
- 用 Softmax + Cross-Entropy 计算 next-token loss；
- 手工推导并更新梯度；
- 观察 loss 下降；
- 最后从一个 token 开始进行自回归采样。

运行命令：

```
python 04_toy_bigram_pretraining.py
```

实验目的

它没有 Attention，也没有 Transformer，但训练目标、Softmax、Cross-Entropy、梯度更新、自回归生成这条主线，与真正 LLM 预训练是一致的。

26. 把真正的 LLM 预训练压缩成 12 步

步骤	发生什么
1	收集原始语料
2	清洗 / 去重 / 过滤
3	Tokenizer 转成 token IDs
4	切片 / Packing 成训练 sequences
5	组成 mini-batch
6	Embedding + Transformer forward
7	每个位置产生 vocabulary logits
8	与 shifted next-token targets 比较
9	Cross-Entropy 得到 loss
10	Backprop 得到 gradients
11	Optimizer 更新 parameters
12	重复大量 steps, 定期验证与保存 checkpoint

最低闭环

如果你能不看材料把这 12 步讲出来，就已经真正理解“大语言模型是怎样被训练出来的”基本过程。

27. 常见误区集中纠正

误区	正确理解
“预训练就是把网页存进模型”	预训练通过梯度更新参数，模型形成分布式统计表示，不是传统数据库写入。
“模型每次只训练一个 token”	训练逻辑是 next-token，但同一序列很多位置可以并行算 loss。
“Loss 只在预测错误时才存在”	Cross-Entropy 对整个概率分布持续提供梯度。
“Base Model 已经等于 ChatGPT 类助手”	通常还需指令微调、偏好对齐和系统层。
“训练 loss 越低，所有能力一定越强”	还需独立评测泛化、推理、安全、鲁棒性等。
“模型完全不记训练数据”	可能存在不同程度 memorization，应与 generalization 同时研究。

28. 与 Agent Systems 的连接：为什么研究 Agent 也需要懂 Pretraining?

- Agent 的“认知底座”通常来自预训练模型，底座能力上限会影响规划、工具调用和错误模式；
- 不同 base model 的训练数据与预训练目标不同，Agent 行为分布也可能不同；
- 模型对代码、网页、工具文档是否熟悉，会影响 benchmark；
- 预训练形成的幻觉、偏差、记忆和知识边界会传递到 Agent 系统；
- 多 Agent 实验不能把所有差异都归因于“协作机制”，底层模型训练差异也是混杂因素。

科研视角

以后你研究 Agent Systems 时，Base Model 不是一个“黑盒常量”。至少要记录模型版本、上下文长度、推理参数，并意识到其预训练历史可能影响结果。

29. 本讲最低闭环测试

- 你能解释 Pretraining 与 Instruction Tuning 的区别吗?
- 你能说清一个文本序列怎样变成多个 next-token 目标吗?
- 你能解释 logits $\rightarrow$ Softmax $\rightarrow$ Cross-Entropy $\rightarrow$ gradient $\rightarrow$ parameter update 吗?
- 你能解释为什么训练能并行, 而生成通常逐 token 吗?
- 你能解释 Base Model 为什么还不是完整聊天助手吗?
- 你能说明 memorization、generalization、contamination 为什么都与预训练有关吗?

如果这些都能回答

你已经不再停留在“LLM 是拿海量数据训练出来的”这句话, 而是有了一个可用于读论文、看训练日志、理解系统问题的具体机制模型。

30. 下一讲：训练数据到底是什么？

本讲主要讲“训练循环”。下一讲会把镜头放到数据端：LLM 的训练数据从哪里来、怎样清洗、去重、过滤、配比，以及数据质量为什么会直接改变模型能力。

- 网页抓取数据为什么既重要又脏？
- 书籍、代码、论文、问答各自提供什么能力信号？
- Deduplication 为什么重要？
- Data Mixture / Sampling Weight 是什么？
- Synthetic Data 为什么越来越重要？
- 数据污染、隐私、版权与安全问题怎样进入训练流程？

下一讲

第14讲：《训练数据：LLM 到底“吃”什么长大？》